CANDIDATE AND JOB MATCHING FOR RECRUITERS

Tracefit architecture

The same person and the same job are described differently, at different times, by sources of different authority. Tracefit keeps every fact with its source, date and exact words, then asks two questions about each person: **can they do this job**, and **how good a fit are they**. Every point of every score traces back to a quote.

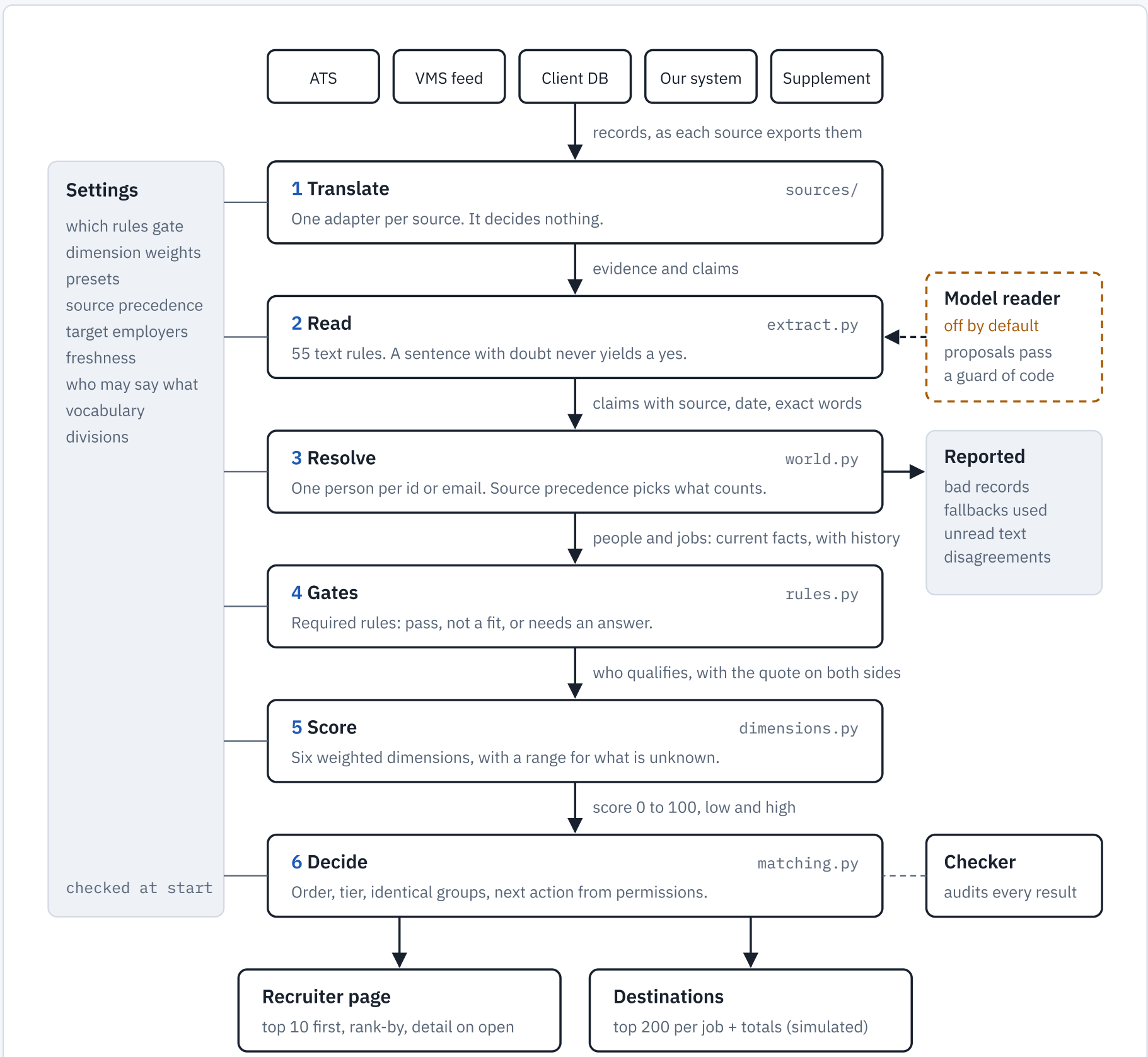
---

5 sources, **3** destinations **10** fit rules, **4** permission rules **6** quality dimensions **231** tests, **110** scenarios timed at **20,000** and **100,000** synthetic people

runs today designed, or built and switched off

How a record becomes a recommendation

Six steps, always in this order. Each hands a plain result to the next, so any one can change without touching the others.

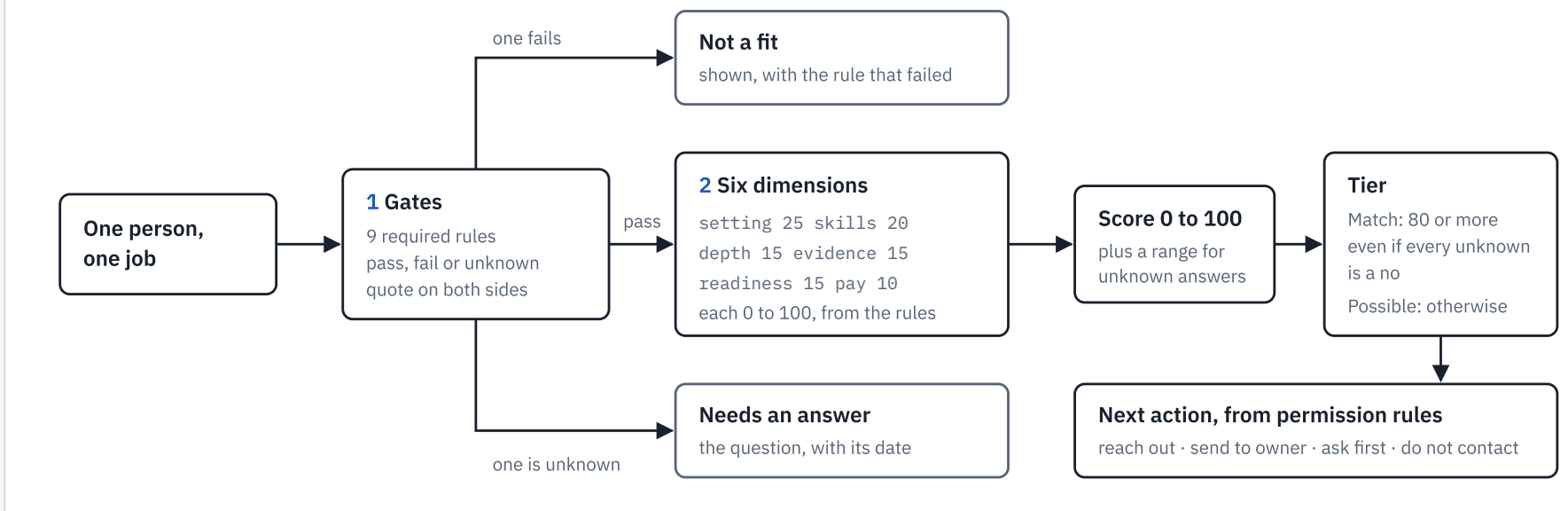


Matching never imports an adapter, and no adapter imports matching; a test reads the imports to keep it that way. A new source is one adapter and one settings block. The model reader is built but switched off: its proposals count only if plain code agrees.

Two questions for every person

First, **can they do this job?** Required rules answer pass, fail or unknown, and no score can buy past them: a missing license or a declined shift is not something a better résumé

makes up for. Then, **how good a fit are they?** The people on the list are then ordered by a score built from six named dimensions. Someone a required rule rules out is not scored.



Permission rules (job open, contact, ownership, requirement source) never change fit; they decide what the recruiter may do. Weights are relative and set per division; a recruiter can also move them on the page, which reorders people inside each list and never moves anyone past a gate.

Setting 25
Dated experience in the job's care setting, or at an employer the client names. Undated: 70.

Skills 20
The tools and certificates the job asks for. A candidate's own "I have not used it" wins; preferred items count half; qualifying by an alternative board route earns two thirds.

Depth 15
Documented years against twice the minimum. Claimed years count at most half.

Evidence 15
Minus 30 for a claim the records do not support or a newer message that could not be read; minus 15 for a stale answer.

Readiness 15
Start date, schedule and shifts met, and how recently the candidate replied.

Pay 10
At or under the approved rate scores 100; 10 points off for each 1% above.

On the sample, the ten people who pass every gate for the home-health job score 81 to 93, each gap traceable to one dimension.

What a recruiter sees

PART OF THE PAGE	WHAT IT ANSWERS
The list	One ranked list per job: everyone who passes every required rule, and everyone a required rule still has a question for, in score order. The top 10 first, then a button for the next 10. A person who needs an answer sits where their score puts them, marked with what is missing. Anyone who must not be contacted is last. People level on every rule are grouped: the page says their order is arbitrary, whether their documents are identical, and what to ask to tell them apart. The people ruled out are one link away.
Rank by	Presets ("Closest to our work", "Experience first", "Can start soonest", "Within budget"), six weight sliders, and filters such as "Can act now" or "Has OASIS". A recruiter can also type their own criterion, a phrase or a school: the list narrows to the people whose own documents say it, matched as whole words, and each card shows the sentence. The whole list re-ranks on the server.
A card	Rank, score and range, one block per rule, the line that sets this person apart, and the next action. The quotes are one click away, in the person panel. Two people with one name show which record each is.
The person panel	Why this score, as a radar and plain words; every rule with its quote; permission; and, folded until opened, the background read from the résumé and everything on file. **Show everything** opens it all.
Compare	Two or three people side by side: radars overlaid, rules row by row with sources, differences highlighted.
How this list was made	Under the list: people on file, people on the list, and people a required rule ruled out. **Show the whole process** opens the points that decided it, the weights drawn as a web, and every step from the sources to the next action, with the count on each rule.
Demo and settings	Replay of the 18:00 snapshot, the 18:05 new job and the 18:10 update; switching between client profiles.

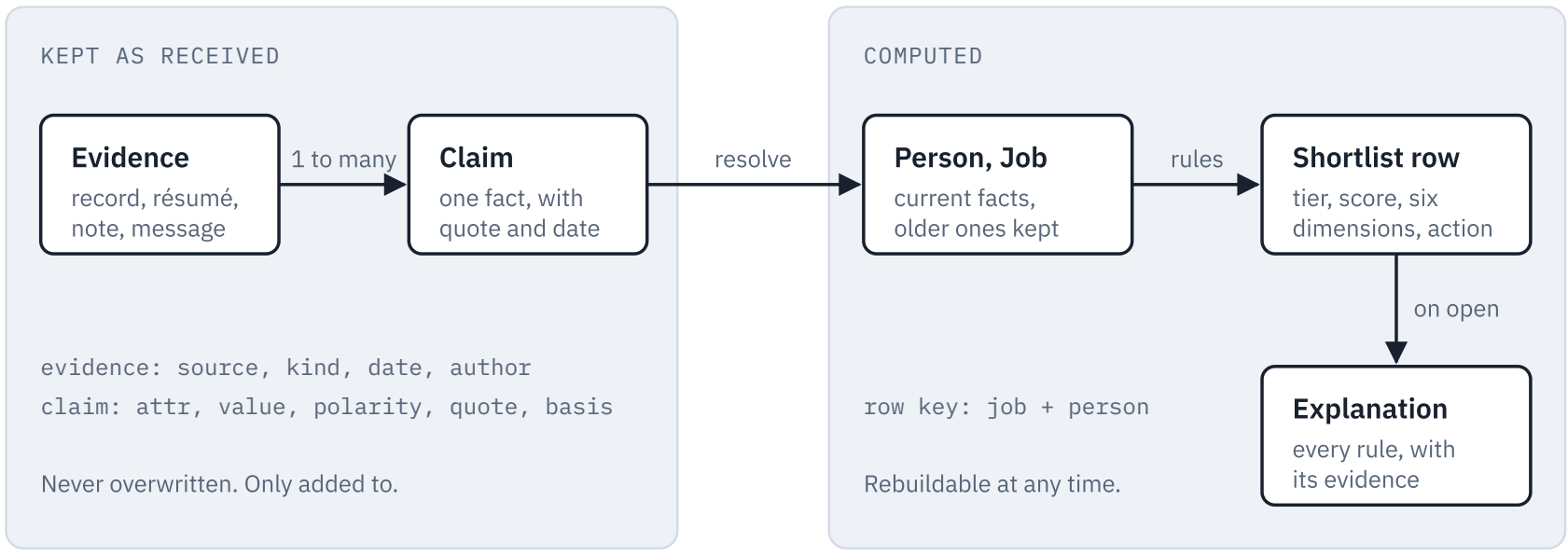
What a client can change without code

SETTING	EXAMPLE
Which rules are gates	Make pay a hard ceiling (the "strict pay" profile)
Dimension weights	Per division: locums put readiness at 30 and pay at 5
Presets	Named weightings a recruiter picks on the page
Target employers	Agencies the client rates highly count as full setting experience
Source precedence	Per fact: "the vendor feed is the authority on start dates". Newest wins by default
Requirement changes from notes	Use and flag, ignore until the feed agrees, or trust; and whose notes count
Freshness windows	Availability older than 30 days is flagged (14 for locums)
Who may say what	A recruiter's note may record availability, never a skill
Thresholds and display	Match at 80, unknown counts half, comparable within 3 points, open each list on its top 10
Vocabulary	Professions, settings, skills, certificates: a new division is configuration

Fixed on purpose: every rule runs for every pair, every fact keeps its source, credentials are never called verified, and nothing is sent automatically.

What is kept, and what is computed

This split is what makes the rest of the design safe. Only two things are the record. Everything a recruiter sees is computed from them and can be thrown away.



The rules are deterministic: the same evidence and settings always give the same rows. So a settings change or a code fix is a rebuild, never a data migration, and an explanation does not need to be stored. A test checks that a rebuilt explanation is identical to one held in full.

What each event costs

Work follows the thing that changed. Times are medians on synthetic people, in one process on an 8 GB laptop, measured before the last changes to the text reader.

WHAT HAPPENS	WORK THAT IS REDONE	GROWS WITH	20,000 PEOPLE	100,000 PEOPLE
A job arrives	Resolve the job, then run all 14 rules for everyone in its profession. Done once.	People in that profession	0.63 s	5.0 s
A candidate changes	Re-read their text, re-resolve them, re-check them against open jobs, replace their one row in each list.	Open jobs in their profession	0.2 ms	1.6 ms
A job's requirements change	Rebuild that job's list, compare with the old one, report only the people who moved.	People in that profession	as a new job	
A job closes	Withdraw its list.	Nothing	not timed	

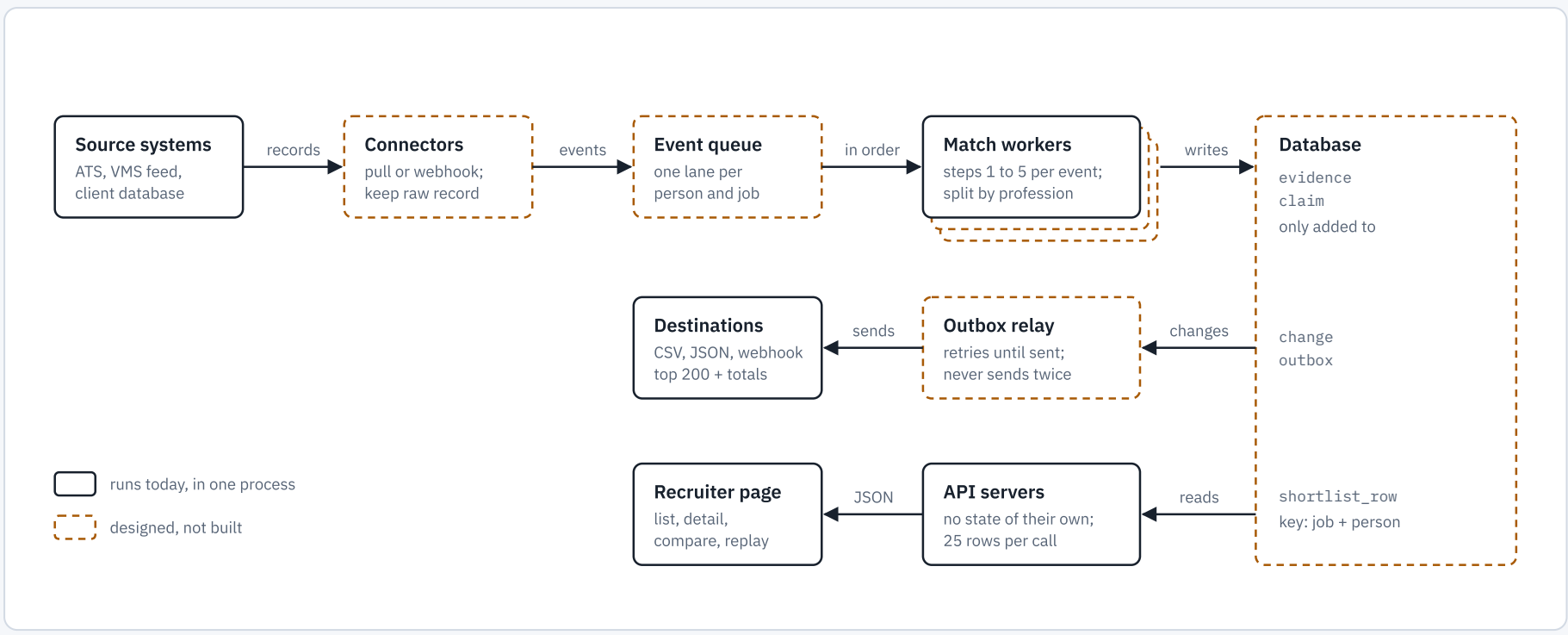
WHAT HAPPENS	WORK THAT IS REDONE	GROWS WITH	20,000 PEOPLE	100,000 PEOPLE
A recruiter opens a list	Read one screenful (10 on the page; timed at 25).	The rows shown	2.6 ms	3.5 ms
A recruiter moves a weight or a filter	Re-score one list from stored dimensions, serve the first screenful. No text is re-read.	The rows in that list	8.1 ms	6.5 ms *
A recruiter opens a person	Rebuild one explanation, with only the evidence it cites.	One person, one job	0.5 ms	0.9 ms
Settings change	Rebuild every open list. Today this is a restart.	Open jobs × their professions	8.3 s for 13 jobs; 3.5 s on four workers	78 s; 8.7 s on sixteen workers (cloud)
The date changes	Freshness, years of experience and start dates all move with the date. Today the reference date is fixed in settings.	Open jobs × their professions	not built	

> Before the 100,000 run the laptop had about 5 GB of swap in use by other apps, so its worst cases include waiting on the disk. The 20,000 run stays under 460 MB, worker runs included, and is the cleaner measurement. * The laptop's 100,000 run predates the single list per job. A clean 100,000 run on a 16-core cloud machine, after that change, gives 51 ms for this row and 6.0 s for a new job on one process.

The one-event path gives the same rows, totals and change messages as rebuilding everything from scratch. That is tested on the sample and on all 110 scenarios. Events that share one timestamp give the same lists in any delivery order; two statements from one person at the same moment that disagree become a question showing both, and a job updated and filled at the same moment is treated as filled.

The same design as a running service

Today the solid boxes run in one process and keep in memory what the dashed boxes would keep on disk. Nothing in the six steps changes. What changes is where records come from and where facts and rows live.



Writes travel along the top, reads along the bottom, and they meet only in the database. A recruiter's page never waits for a shortlist to be built. The three simulated destinations write local files today and send nothing anywhere.

PART	TODAY	AS A SERVICE	WHERE IT PLUGS IN
Intake	Reads the export files named in settings	A connector per source. It saves the raw record first, then hands it to the adapter	An adapter already takes one record at a time
Events	A list replayed in time order	A queue that keeps each person's and each job's events in order	`Live.ingest(event)`
Facts	In memory. A restart re-reads every source	Two tables, only added to. Text is read once per document, not once per start	`World.apply(batch)`

PART	TODAY	AS A SERVICE	WHERE IT PLUGS IN
Rows	In memory, for every open job	**One table keyed by job and person**, with a stored sort key. Only lists being viewed are in memory	`Shortlist.page`, `Shortlist.patch`
Workers	One process by default. Worker processes on one machine are built and measured: at 100,000 people, 6× faster on sixteen cores for one job and 9× for a batch of jobs	**Several machines, split by profession.** A list is independent per job, and per person within a job	`workers.py`, `Shortlist` build
Notifications	Written to a local file	**An outbox table and a relay** that retries and carries a key so a destination can ignore repeats	`sinks/`
Clock	A fixed reference date in settings	**A nightly "day changed" event** that rebuilds open lists	`World(as_of=…)`
Access	No sign-in	**Sign-in, roles, and a log** of who changed a setting or published a list	Server

Six rules that keep it correct across machines

1. **One person, one worker.** Identity is settled before routing, on an index of emails and explicit references, so two records of one person always reach the same place.
2. **Unknown is not "no".** Only a stated, different profession sets a person aside. People with no stated profession stay in every worker's pool.
3. **Order per person and per job.** A late arrival is still placed correctly, because every fact carries its own date.
4. **An event applied twice changes nothing.** Events carry an id; a row is written by its key.
5. **A row, its change message and its outbox entry are written together**, so a recruiter is never told about a move that was not saved.
6. **Every row records the settings version that produced it.** A rebuild writes the new version beside the old one and switches when complete.

> Rules 3 and 4 hold in the code today. Rules 1, 2, 5 and 6 describe how the unbuilt parts must behave; rule 2 is already how the profession index works inside one process.

How large it gets

QUANTITY	FIGURE	BASIS
Evidence items and dated facts for 100,000 people	`460,064` and `1,654,770`	Measured
Rows held for one job with 30,047 people in its profession	`30.4 MB`	Measured
Rows held for 13 open jobs	`354 MB`	Measured
One person against one job, all rules and dimensions	`120 to 220 µs`	Measured
Rows for 1,000 open jobs in one profession of 30,000	`30 million`	Arithmetic. A table, not memory
A candidate update with 1,000 open jobs in their profession	`about 0.2 s`	Estimate from the cost per pair
Rebuilding 1,000 open lists after a settings change	`about 80 min of one core`	Estimate. Divides across workers
One new job (6,126 people) on 1, 2 and 4 worker processes	`0.78 s, 0.50 s, 0.30 s`	Measured at 20,000 people, one machine. Lists identical
All 13 open jobs rebuilt on 1, 2 and 4 worker processes	`8.3 s, 5.2 s, 3.5 s`	Measured at 20,000 people, one machine
Memory of each worker process	`351 MB`	Measured at 20,000 people
One new job at 100,000 people (30,047 in its profession) on 1, 4 and 16 workers	`7.1 s, 2.6 s, 1.17 s`	Measured on a 16-core cloud machine. Lists identical
All 13 open jobs rebuilt at 100,000 people on 1, 4 and 16 workers	`78 s, 24 s, 8.7 s`	Measured, same machine. Nine times faster on sixteen

QUANTITY	FIGURE	BASIS
Memory of each worker process at 100,000 people	about 2.5 GB	Measured. Includes memory shared with the loaded process

A new job is the expensive event, at about 0.15 ms per person in its profession. It happens once per job and off the page's path. Split between worker processes on one machine it shortens, with an identical list: at 100,000 people on a 16-core cloud machine, 7.1 s on one worker and 1.17 s on sixteen. That is measured; across several machines it is not.

Safety nets

- **Unread rather than misread.** A sentence with a negation, doubt or question the reader cannot place never yields a yes, and neither does a passing mention ("my colleague handles OASIS", "held a license until 2023"). A sweep of 163 reworded sentences has no unsafe reading today; it had 57 when first run, and an independent review later found about forty more, now fixed and kept as tests. The reader is pattern-based, so other wordings will still be missed. A newer message that cannot be read turns an older yes into a question.
- **One person, one id.** Every person is keyed by source and record id. Records join only on an explicit reference or the same personal email, never a name, a shared `info@` mailbox or a placeholder such as `N/A`. Two people with one name stay two, and the page shows which record each is.
- **Bad data never stops matching.** A missing file, broken record, unreadable or impossible date, unknown event or duplicate is worked around and listed for a person to fix. A contact restriction nobody could read blocks contact, and a mistyped setting stops the tool with a message.
- **The model reader cannot raise anyone.** Against a stand-in that says yes to everything, no false yes gets through, no contact request is lifted, and nobody's tier rises. **Built, switched off, never run against a real model.**
- **The checker** (`python3 -m shortlist audit`) re-derives each tier and score and checks every quote, permission and the list order. Clean on all 176 sample results, both client profiles, every scenario and with the lying model on; a test plants twelve kinds of mistake and each is caught. It checks the product against its own rules, not whether the rules are right.
- **The worked example cannot drift.** It is generated by the tool, and a test fails if the document stops matching.

Order to build the rest

Each step is useful on its own. More workers come last because the measurements say one process already carries 100,000 people.

1. **Store evidence and claims in a database.** A restart stops re-reading every document.
2. **Store rows in a table and read pages from it.** Memory no longer grows with open jobs.
3. **Add the queue and one real connector.** Replay a real day of events and record the time from event to shortlist.
4. **Add sign-in, the change log and settings versions.** The ownership rule can then use the signed-in recruiter.
5. **Add the nightly date event.**
6. **Split workers by profession,** only if step 3 shows a new job takes too long.

Tied to the sample, and first to revisit with real data: the text reader's 55 rules, the six meanings it recognises in job notes, the vocabulary, and the dimension weights, which are reasoned defaults until they can be calibrated on submissions and placements. The synthetic people measure speed and memory only; they say nothing about match quality.